Encapsulation:
All variables/method declared with ~, #, - are encapsulation (data protection)
example: ~ package-private ~ methods, can only be changed/called within the package, UI or controllers are not able to bypass.
(Table.seat/free, WaitingQueue.enqueue/ remove, Order.markPaid/cancel)

Composition:
Restaurant creates and owns its tables, queue, ledger and staff.
Customer creates new order only when placeOrder is called

Polymorphism:
Customer.placeOrder() and applyWaitingPenalty() call the abstract hooks chooseDish / computeBillCents / getWaitingPenalty.
GameController never checks concrete class
<<Overriding>>: describleDuty() declared as abstract in Staff and each role(waiter, Cashier, Chef) has their own implementation

Legend

Generalization (extends)
Composition (whole creates / owns part)
Aggregation (part created elsewhere)
Association, navigable (field reference)
Dependency («create» / «use»)

static *italic = abstract* + public - private # protected ~ package

Colours: blue = boundary, amber = control, green = entity, grey = enum / utility

Boundary Object

Main
+ main(args: String[]): void

«create»

ConsoleUI
- controller: GameController {readOnly}
- formatter: StateFormatter {readOnly}
- in: Scanner {readOnly}
- out: PrintStream {readOnly}
- echoInput: boolean {readOnly}
- inputEnded: boolean

+ ConsoleUI(controller: GameController, formatter: StateFormatter, input: InputStream, output: PrintStream, echoInput: boolean)
+ run(): void
- playOneTurn(): void
- chooseWaiterTask(): void
- seatCustomer(): boolean
- takeOrder(): boolean
- serveOrder(): boolean
- chooseChefTask(): void
- pickCustomer(prompt: String): Customer
- pickTable(): Table
- readChoice(min: int, max: int): int

1 — formatter — 1

StateFormatter
- RULE: String {readOnly}

+ turnHeader(turn: int, maxTurns: int): String
+ arrival(c: Customer): String
+ startOfTurn(controller: GameController): String
+ customerLine(c: Customer): String
+ orderLine(order: Order): String
+ endOfTurn(report: TurnReport, controller: GameController): String
+ endOfTurnSummary(controller: GameController): String
+ finalResult(result: GameResult, controller: GameController): String
+ average(ledger: Ledger): String
- waiterOptions(r: Restaurant): String
- chefOptions(r: Restaurant): String
- ids(customers: List<Customer>): String
- tableIds(tables: List<Table>): String

Control Object

controller 1 — result — 0..1

GameConfig
- maxTurns: int {readOnly}
- tableCount: int {readOnly}
- arrivalTurns: List<Integer> {readOnly}
- targetPaidCustomers: int {readOnly}
- targetRevenueCents: int {readOnly}
- targetAverageSatisfaction: int {readOnly}

+ GameConfig(maxTurns: int, tableCount: int, arrivalTurns: List<Integer>, targetPaidCustomers: int, targetRevenueCents: int, targetAverageSatisfaction: int)
+ base(): GameConfig

config 1 — 1

GameController
- config: GameConfig {readOnly}
- restaurant: Restaurant {readOnly}
- schedule: ArrivalSchedule {readOnly}
- evaluator: ObjectiveEvaluator {readOnly}
- turn: int
- phase: Phase
- result: GameResult

+GameController(config: GameConfig, restaurant: Restaurant, schedule: ArrivalSchedule, evaluator: ObjectiveEvaluator)
+ startTurn(): List<Customer>
+ waiterSeat(customer: Customer, table: Table): void
+ waiterTakeOrder(customer: Customer): Order
+ waiterServe(customer: Customer): void
+ waiterWait(): void
+ getMaxTurns(): int
+ chefPrepare(order: Order): void
+ chefWait(): void
+ endTurn(): TurnReport
+ isGameOver(): boolean
- requirePhase(expected: Phase): void

«create» — 1

TurnReport
- turn: int {readOnly}
- paidCustomer: Customer {readOnly}
- paidAmountCents: int {readOnly}
- finishedEating: List<Customer> {readOnly}
- departures: List<Customer> {readOnly}
- finalResult: GameResult {readOnly}

+ TurnReport(turn: int, paidCustomer: Customer, paidAmountCents: int,finishedEating: List<Customer>, departures: List<Customer>, finalResult: GameResult)

finalResult 0..1 — 0..1

GameResult
- paidTargetMet: boolean {readOnly}
- revenueTargetMet: boolean {readOnly}
- satisfactionTargetMet: boolean {readOnly}

+ GameResult(paid, revenue, satisfaction: boolean)
+ isVictory(): boolean

paidCustomer [0..1], finishedEating, departures

config 1 — 1

ObjectiveEvaluator
- config: GameConfig {readOnly}

+ ObjectiveEvaluator(config: GameConfig)
+ evaluate(ledger: Ledger): GameResult

evaluator 1 — 1

phase 1 — 1

«enumeration»
GameController::Phase
BETWEEN_TURNS
WAITER
CHEF
RESOLVE
GAME_OVER

ArrivalSchedule
- arrivalTurns: List<Integer> {readOnly}
- factory: CustomerFactory {readOnly}
- nextNumber: int

+ ArrivalSchedule(arrivalTurns:List<Integer>, factory: CustomerFactory)
+ arrivalsFor(turn: int): List<Customer>

schedule 1 — 1

factory 1 — 1

CustomerFactory
+ create(number: int, arrivalTurn: int): Customer

«create»

«use»

Entity Object

customers 0..* — 0..*

restaurant 1 — 1

Customer
+ MAX_SATISFACTION : int = 100 {readOnly}
- number: int {readOnly}
- arrivalTurn: int {readOnly}
- satisfaction: int
- state: CustomerState
- order: Order
- servedTurn: int
- readyToPayTurn: int
- paidSatisfaction: int

Customer(number: int, arrivalTurn: int)
+ chooseDish(menu: Menu): MenuItem {abstract}
+ getWaitingPenalty(): int {abstract}
+ computeBillCents(item: MenuItem): int {abstract}
+ getTypeName(): String {abstract}
+ seat(): void
+ placeOrder(menu: Menu): Order
+ receiveFood(turn: int): void
+ finishEatingIfDone(turn: int): boolean
+ canPay(turn: int): boolean
+ pay(turn: int): int
+ isLosingSatisfaction(): boolean
+ applyWaitingPenalty(): void
+ shouldLeave(): boolean
+ leave(): void
+ isActive(): boolean
+ getId(): String

occupant 0..1 — 0..1

Table
- number: int {readOnly}
- occupant: Customer

+ Table(number: int)
+ isFree(): boolean
+ getId(): String
~ seat(customer: Customer): void
~ free(): void

tables * — 1

customers 0..* — queue 1 — 1

WaitingQueue
- customers: List<Customer> {readOnly}

~ enqueue(customer: Customer): void
~ remove(customer: Customer): boolean
+ contains(customer: Customer): boolean
+ isEmpty(): boolean
+ asList(): List<Customer>

Restaurant
- menu: Menu {readOnly}
- tables: List<Table> {readOnly}
- queue: WaitingQueue {readOnly}
- customers: List<Customer> {readOnly}
- ledger: Ledger {readOnly}
- waiter: Waiter {readOnly}
- chef: Chef {readOnly}
- cashier: Cashier {readOnly}

+ Restaurant(menu: Menu, tableCount: int)
+ admit(customer: Customer): void
+ seat(customer: Customer, table: Table): void
+ takeOrder(customer: Customer): Order
+ serve(customer: Customer, turn: int): void
+ collectPayment(customer: Customer, turn: int): int
+ depart(customer: Customer): void
- freeTableOf(customer: Customer): void
+ findTableOf(customer: Customer): Optional<Table>
+ getFreeTables(): List<Table>
+ getSeatedWithoutOrder(): List<Customer>
+ getCustomersWithReadyOrders():List<Customer>
+ getCookableOrders(): List<Order>
+ getOpenOrders(): List<Order>
+ getPayableCustomers(turn: int): List<Customer>
+ getActiveCustomers(): List<Customer>
+ getQueuedCustomers(): List<Customer>

waiter 1 — 1

chef 1 — 1

cashier 1 — 1

Entity Object

Staff

Waiter
+ Waiter(name: String)
+ describeDuty(): String
+ seat(restaurant: Restaurant, customer: Customer, table: Table): void
+ takeOrder(restaurant: Restaurant, customer: Customer): Order
+ serve(restaurant: Restaurant, customer: Customer, turn: int): void

Chef
+ Chef(name: String)
+ describeDuty(): String
+ prepare(o: Order): void

Cashier
+ Cashier(name: String)
+ describeDuty(): String
+ processPayment(restaurant: Restaurant, turn: int): Optional<Customer>

Staff
(abstract)
- name: String
- actedThisTurn: boolean

Staff(name: String)
+ describeDuty(): String {abstract}
+ resetForTurn(): void
+ waitTurn(): void
ensureAvailable(): void
markActed(): void
+ hasActedThisTurn(): boolean

ledger 1 — 1

menu 1 — 1

Ledger
- revenueCents: int
- paidCount: int
- paidSatisfactionSum: int
- servedCount: int
- departureCount: int

~ recordPayment(amountCents: int, satisfactionAtPayment: int): void
~ recordServed(): void
~ recordDeparture(): void
+ hasPaidCustomers(): boolean
+ getAverageSatisfaction(): double
+ averageSatisfactionAtLeast(target: int): boolean

state 1 — 1

«enumeration»
CustomerState
WAITING
SEATED
AWAITING_FOOD
EATING
READY_TO_PAY
PAID
LEFT

order (Order.customer: back reference) 0..1 — 1

RegularCustomer
- WAITING_PENALTY: int = 8 {readOnly}

+ RegularCustomer(number: int, arrivalTurn: int)
+ chooseDish(menu: Menu): MenuItem
+ getWaitingPenalty(): int
+ computeBillCents(item: MenuItem): int
+ getTypeName(): String

Order
- customer: Customer {readOnly}
- item: MenuItem {readOnly}
- priceCents: int {readOnly}
- unitsDone: int
- status: OrderStatus

+ Order(customer: Customer, item: MenuItem, priceCents: int)
+ prepareOneUnit(): void
+ markServed(): void
~ markPaid(): void
~ cancel(): void
+ isCookable(): boolean
+ isReady(): boolean
+ describeStatus(): String

item * — 1

MenuItem
- name: String {readOnly}
- priceCents: int {readOnly}
- prepUnits: int {readOnly}

+ MenuItem(name: String, priceCents: int, prepUnits: int)
+ toString(): String

items 1..* — 1

Menu
- items: List<MenuItem> {readOnly}

+ Menu(items: List<MenuItem>)
+ standard(): Menu
+ getItems(): List<MenuItem>
+ get(index: int): MenuItem
+ size(): int

status 1 — 1

«enumeration»
OrderStatus
PLACED
IN_PREPARATION
READY
SERVED
PAID
CANCELLED

Money
«final»
- Money()
+ format(cents: int): String

InvalidActionException
- serialVersionUID: long = 1L {readOnly}

+ InvalidActionException(reason: String)